OTERIA CYBER SCHOOL



Les fondamentaux de la programmation

Objectif

- Comprendre les sockets réseaux
- Appliquer les sockets à Python 3
- Créer un Client puis un Client/serveur





Chapitre 1

- Des clients
- Un serveur => plusieurs services :
  - FTP (21)
  - HTTP (80)
  - HTTPS (443)
  - SSH (22) ….

sudo cat /etc/services

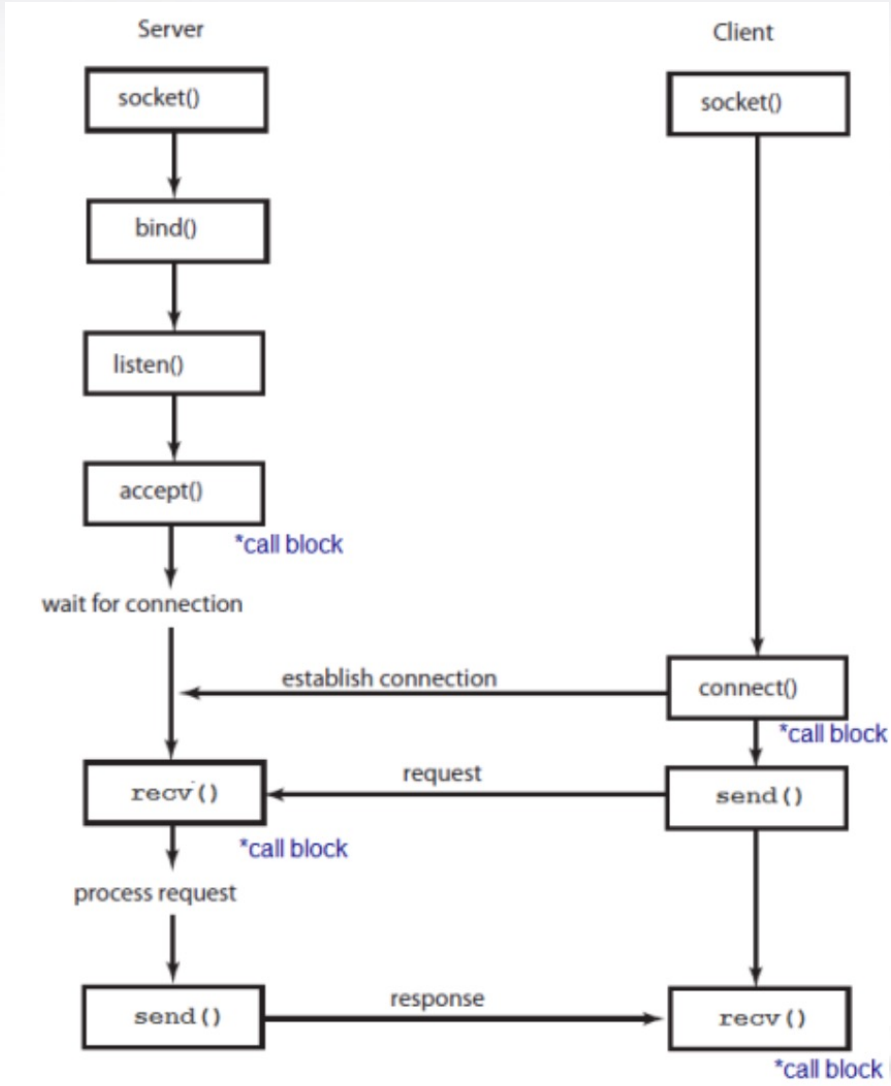
Qu’est ce qu’un socket

- Le socket est donc dans notre cas **une association au niveau de l'OS entre un programme qui tourne en boucle et le port de la machine** qui lui a été dédié. On dit d'ailleurs que le programme écoute le port qui lui a été réservé. Il écoute le port et répond aux demandes faites par ce port.

- Socket = Adresse Ip + port



Principe en Python





Serveur simple en Python

```
2
3  import socket
4
5  socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6  socket.bind(('', 1337))
7
8  while True:
9          socket.listen(5)
10         client, address = socket.accept()
11         print ("{} connected".format( address ))
12
13         response = client.recv(255)
14         if response != "":
15                 print (response)
16
17 print ("Close")
18 client.close()
19 stock.close()
```

OSError: [Errno 48] Address already in use
No specific suggestions for this error (yet).
Let Thonny develo

Shell

```
  File "/Users/franckebel/Documents/formati
erveur.py", line 6, in <module>
    socket.bind(('', 15555))
OSError: [Errno 48] Address already in use

>>> %Run serveur.py

('127.0.0.1', 51773) connected
b'test\n'
```

exercices — nc localhost 1337 — 59×15

```
./exercice3_TD7.sh: line 1: syntax error near unexpected t
oken `;'
./exercice3_TD7.sh: line 1: `;exec < $1'
francks-MBP:exercices franck_ebel$ nano exercice3_TD7.sh
francks-MBP:exercices franck_ebel$ ./exercice3_TD7.sh text
.txt
=> ligne1
=> ligne2
=> ligne3
=> ligne4
francks-MBP:exercices franck_ebel$ nano exercice3_TD7.sh
francks-MBP:exercices franck_ebel$ nc localhost 15555
francks-MBP:exercices franck_ebel$ nc localhost 1337
test
```



Client simple en Python

```python
# coding: utf-8

import socket,time

hote = "localhost"
port = 1337

socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
socket.connect((hote, port))
print ("Connection on {}".format(port))
time.sleep(0.5)
data=socket.recv()
print(data.decode())
socket.send("Salut, mon nom est Franck!".encode())
time.sleep(0.5)

print ("Close")
socket.close()
```



Test client/serveur

reseau_python — Python serveur.py — 63×12

/Users/franckebel/Documents/Mes images — -bash ... ...icence_CDAISI/python/reseau_python — Python serveur.py

```
francks-MBP:reseau_python franck_ebel$ python serveur.py
('127.0.0.1', 52372) connected
Salut, mon nom est Franck!
('127.0.0.1', 52375) connected
Salut, mon nom est Franck!
```

reseau_python — -bash — 80×11

```
Close
francks-MacBook-Pro:reseau_python franck_ebel$ python client.py
Connection on 1337
Close
francks-MacBook-Pro:reseau_python franck_ebel$ python client.py
Connection on 1337
Close
francks-MacBook-Pro:reseau_python franck_ebel$ python client.py
Connection on 1337
Close
francks-MacBook-Pro:reseau_python franck_ebel$
```



Dir(socket)

```
 '_socket', 'close', 'create_connection', 'dup', 'errno', 'erro
r', 'fromfd', 'gaierror', 'getaddrinfo', 'getdefaulttimeout', '
getfqdn', 'gethostbyaddr', 'gethostbyname', 'gethostbyname_ex',
 'gethostname', 'getnameinfo', 'getprotobyname', 'getservbyname
', 'getservbyport', 'has_ipv6', 'herror', 'htonl', 'htons', 'if
_indextoname', 'if_nameindex', 'if_nametoindex', 'inet_aton', '
inet_ntoa', 'inet_ntop', 'inet_pton', 'io', 'ntohl', 'ntohs', '
os', 'selectors', 'setdefaulttimeout', 'sethostname', 'socket',
 'socketpair', 'sys', 'timeout']
```



Méthode des sockets

https://docs.python.org/3/library/socket.html

Exceptions

exception socket.**error**

A deprecated alias of OSError.

Changed in version 3.3: Following **PEP 3151**, this class was made an alias of OSError.

exception socket.**herror**

A subclass of OSError, this exception is raised for address-related errors, i.e. for functions that use *h_errno* in the POSIX C API, including gethostbyname_ex() and gethostbyaddr(). The accompanying value is a pair (h_errno, string) representing an error returned by a library call. *h_errno* is a numeric value, while *string* represents the description of *h_errno*, as returned by the hstrerror() C function.

Changed in version 3.3: This class was made a subclass of OSError.

exception socket.**gaierror**

A subclass of OSError, this exception is raised for address-related errors by getaddrinfo() and getnameinfo(). The accompanying value is a pair (error, string) representing an error returned by a library call. *string* represents the description of *error*, as returned by the gai_strerror() C function. The numeric *error* value will match one of the EAI_* constants defined in this module.

Changed in version 3.3: This class was made a subclass of OSError.

exception socket.**timeout**

A subclass of OSError, this exception is raised when a timeout occurs on a socket which has had timeouts enabled via a prior call to settimeout() (or implicitly through setdefaulttimeout()). The accompanying value is a string whose value is currently always “timed out”.



Socket error

```
# First try-except block -- create socket
try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
except socket.error, e:
    print "Error creating socket: %s" % e
    sys.exit(1)

# Second try-except block -- connect to given host/port
try:
    s.connect((host, port))
except socket.gaierror, e:
    print "Address-related error connecting to server: %s" % e
    sys.exit(1)
except socket.error, e:
    print "Connection error: %s" % e
    sys.exit(1)
```

```
# Third try-except block -- sending data
try:
    s.sendall("GET %s HTTP/1.0\r\n\r\n" % filename)
except socket.error, e:
    print "Error sending data: %s" % e
    sys.exit(1)

while 1:
    # Fourth tr-except block -- waiting to receive data from remote host
    try:
        buf = s.recv(2048)
    except socket.error, e:
        print "Error receiving data: %s" % e
        sys.exit(1)
    if not len(buf):
        break
    # write the received data
    sys.stdout.write(buf)
```



Programmation parallele avec Threading

- Jusqu'à présent nous avons effectué des programmes « linéaires » :
  - **Action par action**
  - **Une action ne commence lorsque l'autre est terminée**

=>Actions en parallèle : Thread



Programme linéaire

```
import random
import sys
import time

# Répète 20 fois
i = 0
while i < 20:
    sys.stdout.write("1")
    sys.stdout.flush()
    attente = 0.2
    attente += random.randint(1, 60) / 100
    # attente est à présent entre 0.2 et 0.8
    time.sleep(attente)
    i += 1
```



Thread

```
class Afficheur(Thread):

    """Thread chargé simplement d'afficher une lettre dans la console."""
    def __init__(self, lettre):
        Thread.__init__(self)
        self.lettre = lettre

    def run(self):
        """Code à exécuter pendant l'exécution du thread."""
        i = 0
        while i < 20:
            sys.stdout.write(self.lettre)
            sys.stdout.flush()
            attente = 0.2
            attente += random.randint(1, 60) / 100
            time.sleep(attente)
            i += 1
```

```
# Création des threads
thread_1 = Afficheur("1")
thread_2 = Afficheur("2")

# Lancement des threads
thread_1.start()
thread_2.start()

# Attend que les threads se terminent
thread_1.join()
thread_2.join()
```

Shell

```
>>> %Run premier_thread.py
 11111111111111111111
>>> %Run deuxieme_thread.py
 12121212121121212121211211221212112222
>>>
```



Reseau et Thread : serveur

```
# coding: utf-8

import socket
import threading

class ClientThread(threading.Thread):

    def __init__(self, ip, port, clientsocket):

        threading.Thread.__init__(self)
        self.ip = ip
        self.port = port
        self.clientsocket = clientsocket
        print("[+] Nouveau thread pour %s %s" % (self.ip, self.port, ))

    def run(self):

        print("Connexion de %s %s" % (self.ip, self.port, ))

        r = self.clientsocket.recv(2048)
        print("Ouverture du fichier: ", r, "...")
        fp = open(r, 'rb')
        self.clientsocket.send(fp.read())

        print("Client déconnecté...")

tcpsock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
tcpsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
tcpsock.bind(("",1111))
```

```
while True:
    tcpsock.listen(10)
    print( "En écoute...")
    (clientsocket, (ip, port)) = tcpsock.accept()
    newthread = ClientThread(ip, port, clientsocket)
    newthread.start()
```



Réseau et Thread : Client

```
# coding: utf-8

import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("", 1111))

print("Le nom du fichier que vous voulez récupérer:")
file_name = input(">> ") # utilisez raw_input() pour les anciennes versions python
s.send(file_name.encode())
file_name = 'data/%s' % (file_name,)
r = s.recv(9999999)
with open(file_name,'wb') as _file:
    _file.write(r)
print("Le fichier a été correctement copié dans : %s." % file_name)
```

